

♦ Android Layout :-

A Layout in Android is a container that decides how UI elements (Views) are arranged and positioned on the screen.

Layout is the structure or container used to arrange UI components such as TextView, Button, EditText, ImageView, etc. on an Android screen.

Example:-

Suppose we want to create a Login Screen:

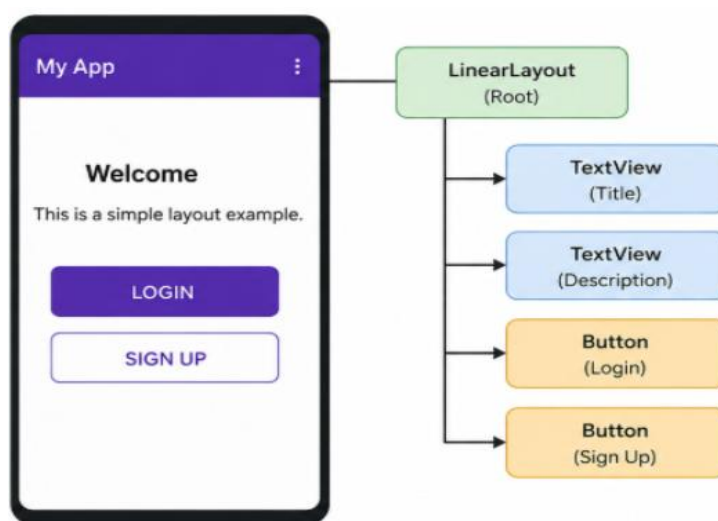
Login -> TextView → displays Login

[Enter Email] -> EditText → accepts Email and Password

[Password] -> accepts Password

[Login] -> Button → displays Login button & when user click on login user will logged in

Here the role of Layout is to decides where these components will appear. Basically the layout is responsible for arranging the position of the elements in the container by maintaining proper spacing and alignment.



1. LinearLayout

Linear Layout arranges UI elements one after another in a single direction.

It can arrange views:

- Vertically → top to bottom
- Horizontally → left to right

It is one of the easiest layouts for beginners.

Main attributes

Attribute	Purpose
android:orientation	Sets vertical or horizontal direction
android:gravity	Positions child elements inside the layout
android:padding	Gives space inside the layout
android:layout_width	Width of layout
android:layout_height	Height of layout
android:weightSum	Helps divide available space
android:layout_weight	Gives proportional space to child views

Example

```
<LinearLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    android:orientation="vertical"
```

```
android:padding="20dp">
```

```
<TextView
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Login"
```

```
    android:textSize="25sp"/>
```

```
<EditText
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:hint="Enter Email"/>
```

```
<Button
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Login"/>
```

```
</LinearLayout>
```

Output UI theory

The UI will appear vertically:

Login

[Enter Email]

[Login]

Because:

```
android:orientation="vertical"
```

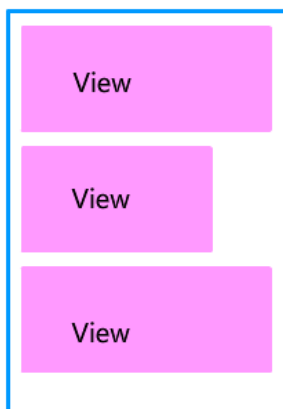
all elements appear below one another.

Use:-

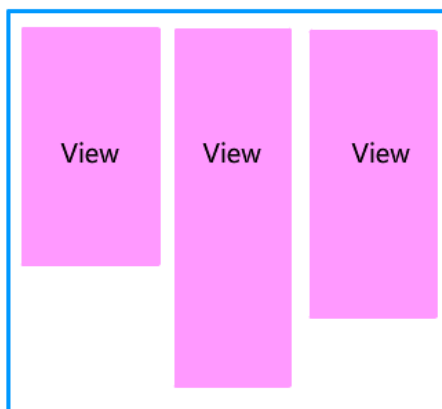
LinearLayout is useful for:

- Login forms
- Registration forms
- Simple menus
- Settings screens
- Button groups

Vertical LinearLayout



Horizontal LinearLayout



2. ConstraintLayout

ConstraintLayout is one of the most important layouts in modern Android development.

Instead of arranging elements only vertically or horizontally, we can position elements by creating constraints between views and their parent layout.

For example:

Button should be below EditText.

TextView should be centered in the screen.

Image should be attached to the top of the screen.

Main attributes

Attribute	Purpose
app:layout_constraintTop_toTopOf	Connects top with another view/parent
app:layout_constraintBottom_toBottomOf	Connects bottom
app:layout_constraintStart_toStartOf	Connects starting side
app:layout_constraintEnd_toEndOf	Connects ending side
app:layout_constraintTop_toBottomOf	Places view below another view
android:layout_marginTop	Adds top spacing
android:layout_marginStart	Adds start spacing
android:layout_width	Width
android:layout_height	Height

Example

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

```
<TextView
    android:id="@+id/title"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Welcome"
    android:textSize="28sp"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    android:layout_marginTop="40dp"/>
```

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Continue"
    app:layout_constraintTop_toBottomOf="@id/title"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
```

```
        android:layout_marginTop="20dp"/>  
</androidx.constraintlayout.widget.ConstraintLayout>
```

Output UI:-

Welcome
[Continue]

Use:-

ConstraintLayout is useful for:

- Login screens
- Dashboards
- Profile screens
- E-commerce screens

3. FrameLayout

FrameLayout is mainly used when we want to place views on top of each other.

Think of it like stacking papers.

The first view is placed at the bottom and later views can appear over it.

Main attributes

Attribute	Purpose
android:layout_width	Width
android:layout_height	Height
android:foreground	Adds foreground content
android:gravity	Controls positioning
android:layout_gravity	Controls position of child views

Example

```
<FrameLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="250dp">
```

```
<ImageView
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

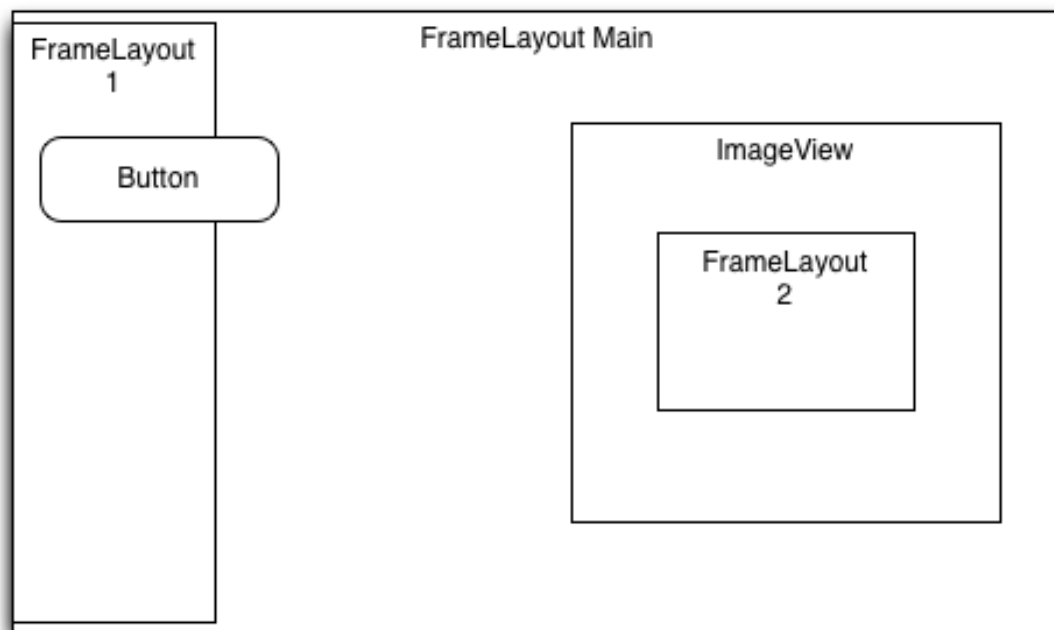
```
    android:src="@drawable/background"
```

```
        android:scaleType="centerCrop"/>
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome"
        android:textSize="30sp"
        android:layout_gravity="center"/>
</FrameLayout>
```

Use:-

FrameLayout is commonly useful for:

- Image + text overlay
- Video player controls
- Loading screen
- Banner designs



4. RelativeLayout :-

RelativeLayout positions UI elements relative to the parent layout or relative to another view.

For example:-

Place button below TextView.

Place image at the right side.

Center TextView inside parent.

It was very commonly used in older Android applications.

Today, ConstraintLayout is generally preferred for more complex interfaces, but RelativeLayout is still useful for students to understand.

Main attributes

Attribute	Purpose
android:layout_below	Places view below another view
android:layout_above	Places view above another
android:layout_toEndOf	Places view after another view
android:layout_toStartOf	Places view before another
android:layout_centerInParent	Centers element
android:layout_alignParentTop	Aligns with parent top
android:layout_alignParentBottom	Aligns with parent bottom

Example:-

```
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:id="@+id/title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome"
        android:textSize="25sp"
        android:layout_centerHorizontal="true"/>

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="StartBTN"
        android:layout_below="@id/title"
        android:layout_centerHorizontal="true"/>
</RelativeLayout>
```

Output :-

Welcome

StartBTN

The button appears below Welcome because:

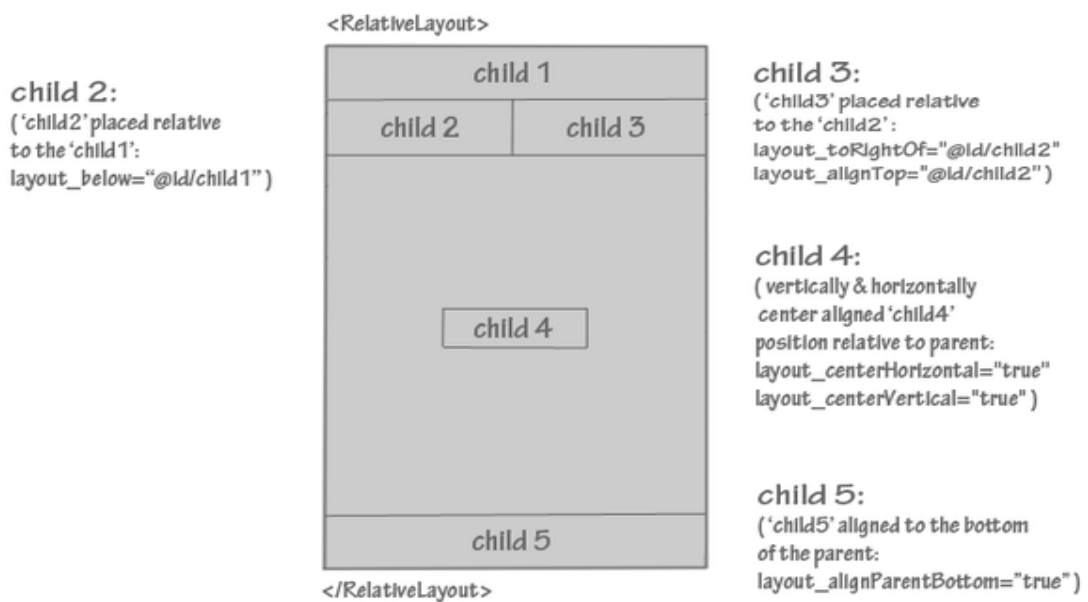
`android:layout_below="@id/title"`

Use:-

Useful for:

- Simple relative positioning
- Small forms
- Older Android projects
- Understanding view relationships

Relative Layout



5. TableLayout

TableLayout arranges information in the form of rows and columns, similar to a table.

Inside it we normally use:

<TableRow>

Example

<TableLayout

xmlns:android="http://schemas.android.com/apk/res/android"

android:layout_width="match_parent"

android:layout_height="wrap_content">

<TableRow>

<TextView

android:text="Name"/>

<TextView

android:text="Marks"/>

</TableRow>

<TableRow>

<TextView

android:text="Rahul"/>

<TextView

android:text="85"/>

</TableRow>

</TableLayout>

Output:-

Name	Marks
Rahul	85

Main attributes

android:stretchColumns

android:shrinkColumns

android:collapseColumns

Uses:-

Useful for:

- Student marks
- Simple reports
- Price comparisons
- Small tabular information

For large or dynamic datasets, RecyclerView is normally more appropriate than TableLayout.

6. Grid Layout:-

GridLayout is an Android layout that arranges UI elements in a grid format using rows and columns.

GridLayout is a layout that arranges UI components in rows and columns, similar to a table or grid.

For example:

```
| 1 | 2 | 3 |  
| 4 | 5 | 6 |  
| 7 | 8 | 9 |
```

This type of arrangement is useful when you want multiple buttons or items to appear in a structured grid.

Main Attributes of GridLayout

Attribute	Purpose
android:rowCount	Specifies number of rows
android:columnCount	Specifies number of columns
android:orientation	Controls how children are placed
android:alignmentMode	Controls alignment of items
android:useDefaultMargins	Adds default margins between items
android:layout_width	Sets layout width
android:layout_height	Sets layout height

Important child attributes

Attribute	Purpose
android:layout_row	Specifies which row the view belongs to
android:layout_column	Specifies which column the view belongs to
android:layout_rowSpan	Makes a view occupy multiple rows
android:layout_columnSpan	Makes a view occupy multiple columns

Example:-

<GridLayout

```
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:columnCount="3"
android:rowCount="3"
android:useDefaultMargins="true">
```

<Button

```
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="1"/>
```

<Button

```
android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"  
    android:text="2"/>
```

```
<Button  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="3"/>
```

```
<Button  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="4"/>
```

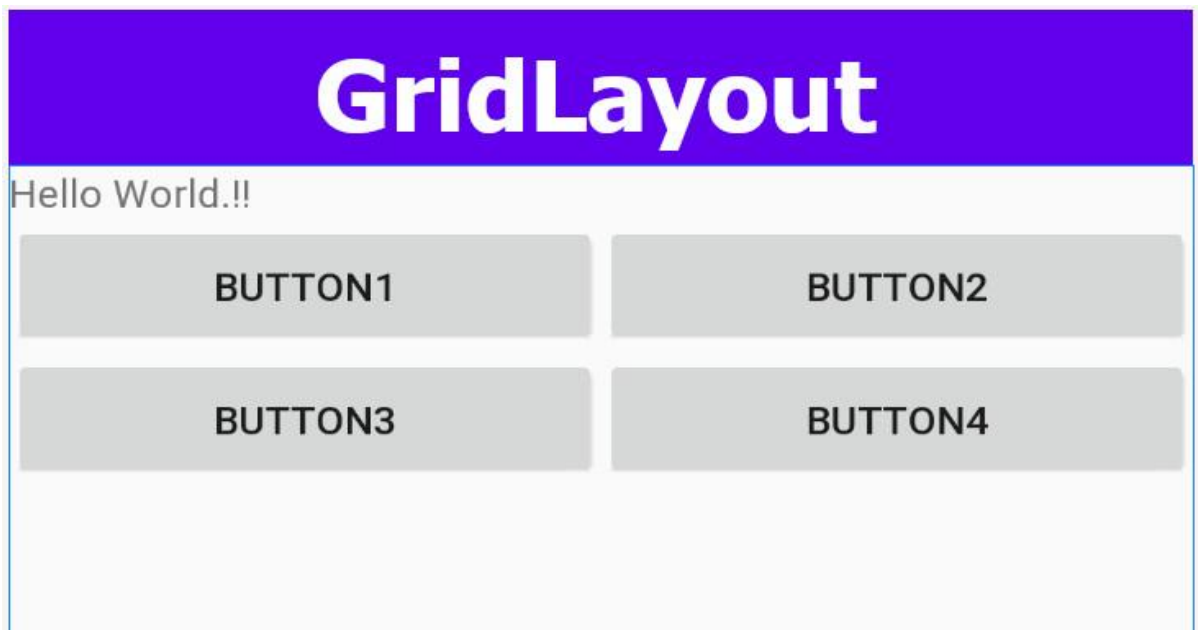
```
<Button  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="5"/>
```

```
<Button  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="6"/>
```

```
</GridLayout>
```

Output:-

	1		2		3	
	4		5		6	



GridLayout vs LinearLayout:-

LinearLayout

Arranges views in one direction

Vertical or horizontal

Good for forms

Example: Login form

GridLayout

Arranges views in rows and columns

Grid structure

Good for grids

Example: Calculator

Difference Between Android Layouts:-

Layout	Main Idea	Arrangement	Best Used For
LinearLayout	Arrange one after another	Row OR Column	Forms, simple screens
RelativeLayout	Position relative to other views	Relative positions	Simple dependent positioning
FrameLayout	Stack views	One over another	Overlays, fragments
GridLayout	Arrange in grid	Rows + Columns	Calculator, categories
TableLayout	Arrange like a table	Rows + Columns	Tabular data
ConstraintLayout	Position using constraints	Flexible 2D positioning	Complex/professional UI

Easy Memory Trick:-

LINEAR → One direction

RELATIVE → Relative position

F2RAME → Stack / Overlay

GRID → Rows + Columns

TABLE → Table

CONSTRAINT → Connected positions